

Practical Exam (2022)

معاك رقمين هتجمعهم لو زوجي هيطلع زي ما هو و لو فردي هتضربه في اثنين يعني تعمل shift left by 1 bit وده سهل.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
entity add_with_shift is
generic (n: natural := 4);
Port (a: in STD_LOGIC_VECTOR (n-1 downto 0);
b: in STD_LOGIC_VECTOR (n-1 downto 0);
res: out STD_LOGIC_VECTOR (n-1 downto 0));
end add_with_shift;
architecture Behavioral of add_with_shift is
signal resu: STD_LOGIC_VECTOR (n-1 downto 0);
begin
resu <= a + b;
res <= resu(n-2 DOWNT0 0) & '0' when resu(0) = '1' ELSE
resu;    --if the least significant bit is '1' then the number is odd.
end Behavioral;
```

الفكرة: هتشوف أول بيت من اليمين لو بتساوي واحد يبقى دا رقم فردي.

معاك رقمين أول رقم هتعمله rotate shift right و الثاني rotate shift left وبعد متعمل كده هتدخلهم علي subtract circuit هتعملهم مينس بعض من الاخر ده كان سؤال اللي امتحنته.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
entity rotate_with_subtraction is
generic (n: natural := 4);
Port (a: in STD_LOGIC_VECTOR (n-1 downto 0);
b: in STD_LOGIC_VECTOR (n-1 downto 0);
res: out STD_LOGIC_VECTOR (n-1 downto 0));
end rotate_with_subtraction;
architecture Behavioral of rotate_with_subtraction is
signal a_sig, bb: STD_LOGIC_VECTOR (n-1 downto 0);
begin
a_sig <= a(0) & a(n-1 downto 1);    --rotated one bit to the right
bb <= not(b(n-2 downto 0) & b(n-1)) + 1;    --rotated to the left
by one bit with 2's complement
res <= a_sig + bb;    --result of their subtraction
end Behavioral;
```

الفكرة: الروتيشن شيفت معناها أنك بتخلي الرقم يلف، يعني البيتس اللي بترميها هي اللي بتلرزها من الناحية الثانية، مش زي الشيفت العادي بتلرز أصفار.

ملحوظة: أنا عامله يشيفت بيت واحدة، فلو أكثر من كذا هتغير عدد البيتس اللي بتحركهم.

معاك رقمين لو الأول أكبر من ثاني هتعمل حاجه ولو أصغر هتعمل حاجه
ثانية.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
entity Test_Less is
generic (n: natural := 4);
Port (a: in STD_LOGIC_VECTOR (n-1 downto 0);
b: in STD_LOGIC_VECTOR (n-1 downto 0);
out_ : out STD_LOGIC);
end Test_Less;
architecture Behavioral of Test_Less is
signal bb, res: STD_LOGIC_VECTOR (n downto 0);
begin
bb <= '1' & (not b) + 1;
res = '0' & a + bb;
out_ <= not(res(n)); -- outputs '0' if a is smaller and '1' otherwise.
end Behavioral;
```

الفكرة: هتطرحهم من بعض ولو الطرح بالسالب (يعني آخر بيت عالشمال
بواحد) فبكدا يبقى الرقم الثاني أكبر لو غير كدا فيبقى الرقم الثاني مش أكبر.
ملحوظة: لو فيه كونديشن أنهم قد بعض يحصل حاجة معينة ففي الحالة دي
هتضيف حالة جديدة وهي أنك تشوف res نفسها، لو بتساوي صفر.

عاوزك تدخل input 1 و input 2 <== تاخذ اول رقمين من علي الشمال
من اول رقم واول رقمين من علي اليمين من ثاني رقم وتلزمهم في بعض
وبعدها تعمل rotate left by 1 ... هتاخذ اول رقم من ع الشمال ويبقى هو
اول واحد من اليمين وكل رقم يشفت 1.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity p1 is
Port (x: in STD_LOGIC_VECTOR (3 downto 0);
y: in STD_LOGIC_VECTOR (3 downto 0);
z: out STD_LOGIC_VECTOR (3 downto 0));
end p1;
architecture Behavioral of p1 is
signal a1: STD_LOGIC_VECTOR (3 downto 0);
signal a2: STD_LOGIC_VECTOR (3 downto 0);
signal a3: STD_LOGIC_VECTOR (3 downto 0);
begin
a1 <= x(3 downto 2) & "00";
a2 <= "00" & y(1 downto 0);
a3 <= a1(3 downto 2) & a2(1 downto 0);
z <= a3(2 downto 0) & a3(3);
end Behavioral;
```

هتدخل input هتعمله مرة shift left ومرة shift right و تجمع ال shifts
مع بعض مرة وتطرحهم مرة بس كدا.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;
entity p2 is
Port (a: in STD_LOGIC_VECTOR (3 downto 0);
b: out STD_LOGIC_VECTOR (3 downto 0);
c: out STD_LOGIC_VECTOR (3 downto 0));
end p2;
architecture Behavioral of p2 is
signal x: STD_LOGIC_VECTOR (3 downto 0);
signal y: STD_LOGIC_VECTOR (3 downto 0);
begin
x <= a(1 downto 0) & "00";
y <= "00" & a(3 downto 2);
b <= x + y;
c <= x + (not y) + 1;
end Behavioral;
```

عاوزك تاخذ الinput و تخرجه بعكس الترتيب اللي دخل بيه.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.all;
entity p3 is
Port (x: in STD_LOGIC_VECTOR (3 downto 0);
y: out STD_LOGIC_VECTOR (3 downto 0));
end p3;
architecture Behavioral of p3 is
signal a: std_logic_vector (3 downto 0);
begin
a(3 downto 0) <= x(3 downto 0);
y <= a(0) & a(1) & a(2) & a(3);
end Behavioral;
```

عاوز تدخل input 2 تعمل للأول شفت ليفت وللتاني شيفت رايت وتطرحهم
من بعض.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;
entity p4 is
Port ( x : in STD_LOGIC_VECTOR (3 downto 0);
y : in STD_LOGIC_VECTOR (3 downto 0);
z : out STD_LOGIC_VECTOR (3 downto 0));
end p4;
architecture Behavioral of p4 is
signal a: STD_LOGIC_VECTOR (3 downto 0);
signal b: STD_LOGIC_VECTOR (3 downto 0);
begin
a <= x(1 downto 0) & "00";
b <= "00" & y(3 downto 2);
z <= a + (not b) + 1;
end Behavioral;
```

لو ال input أكبر من 8 تجمع عليه 2 و لو الناتج أكبر من 15 تخلي
ال carryout ب1.. و لو ال input أقل من 8 تعمل rotate right و تخلي
ال carryout ب0.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_1164.UNSIGNED;
entity question is
Port (a: in STD_LOGIC_VECTOR (3 downto 0);
x: out STD_LOGIC_VECTOR (3 downto 0);
carry: out STD_LOGIC);
end question;
architecture Behavioral of question is
signal temp: std_logic_vector (4 downto 0);
begin
temp <= '0' & a + "00010" when a >= "1000" else '0' & a(0) & a(3
downto 1);
carry<= temp(4);
x<= temp(3 downto 0);
end Behavioral;
```

Another solution:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;
entity practical is
Port ( i : in STD_LOGIC_VECTOR (3 downto 0);
o : out STD_LOGIC_VECTOR (3 downto 0);
carryOut : out STD_LOGIC);
end practical;
architecture Behavioral of practical is
begin
o <= i+2 when i >= "1000" else
i(0) & i(3 downto 1);
carryOut <= '1' when (i >= "1000" and i+2 >= "1111") else
'0';
end Behavioral;
```

Question:

Implement this Module that can:

1. Rotate Data1 1 bit to the left.
2. Rotate Data2 1 bit to the right.
3. Apply the following operation (Data1 - Data2) on the rotated data.

The Implement Module has the following inputs/outputs:

- Data1: 4-bit vector containing input 1.
- Data2: 4-bit vector containing input 2.
- Output: 4-bit vector containing the subtraction result.

Control Word:

Data1 (4 bits)	Data2 (4 bits)

• Simulation:

- Input = 10100101 → output = 1011
- Input = 11110001 → output = 0111
- Input = 10011101 → output = 0101

• Requirement:

- Implement module.
- Test bench.

Test process:

```
-- Stimulus process
stim_proc: process
begin
    -- hold reset state for 100 ns.
    Data1 <= "1010";
    Data2 <= "0101";

    wait for 100 ns;
    Data1 <= "1111";
    Data2 <= "0001";

    wait for 100 ns;
    Data1 <= "1001";
    Data2 <= "1101";

    wait;
end process;
```

Hint:

You will need to use 'use IEEE.STD_LOGIC_UNSIGNED.ALL;' to be able to use '+' operator.

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity Q9 is

Port (Data1 : in STD_LOGIC_vector (3 downto 0);

Data2 : in STD_LOGIC_vector (3 downto 0);

Output1 : out STD_LOGIC_vector (3 downto 0));

end Q9;

architecture Behavioral of Q9 is

signal x: STD_LOGIC_vector (3 downto 0);

signal y: STD_LOGIC_vector (3 downto 0);

begin

x <= Data1(2 downto 0) & Data1(3);

y <= Data2(0) & Data2(3 downto 1);

Output1 <= x + not(y) + 1;

end Behavioral;

معاك 3 inputs و عايزني اجمع اول اتنين منهم و بعدها اقارن الناتج بالتالت و لو الناتج اكبر من التالت.. اطلع ال output ب1 و لو الناتج اصغر من التالت اطلعه ب0.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;
entity training is
Port ( in1,in2,in3: in std_logic_vector (3 downto 0);
result: out STD_LOGIC);
end training;
architecture Behavioral of training is
signal temp: std_logic_vector (4 downto 0);
begin
temp <= ('0' & in1) + ('0' & in2);
result <= '1' when (temp > in3) else '0';
end Behavioral;
```

Another solution:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
entity t is
    Port ( d1: in STD_LOGIC_VECTOR (3 downto 0);
          d2: in STD_LOGIC_VECTOR (3downto 0);
          d3: in STD_LOGIC_VECTOR (3 downto 0);
          s: out STD_LOGIC);
end t;
architecture Behavioral of t is
    Signal add: STD_LOGIC_VECTOR (3 downto 0);
    Signal comm: STD_LOGIC VECTOR (4 downto 0);
begin
    add <= d1 + d2;
    comm <= ('0' & add) + ('0' & not(d3)) + 1;
    s <= comm(4);
end Behavioral;
```

جالي اني هاخذ 2 input واعمل concatenate لل upper part من input1 مع ال lower part من input2 وبعدين الي يطلع دا... مره اعمله shift left ومره اعمله shift right by 1 bit وبعدين اطرحهم من بعض.

Upper part: اول اثنين من الشمال

Lower Part: اول اثنين من اليمين

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

```
entity totestonit is
```

```
    Port(data1: in STD_LOGIC_VECTOR (3 downto 0);
```

```
          data2: in STD_LOGIC_VECTOR (3 downto 0);
```

```
          output: out STD_LOGIC_VECTOR (3 downto 0));
```

```
end totestonit;
```

```
architecture Behavioral of totestonit is
```

```
    signal con_data, shiftright, shiftright, sub: STD_LOGIC_VECTOR(3  
downto 0);
```

```
begin
```

```
    con_data <= data1(3 downto 2)& data2(1 downto 0);
```

```
    shiftright <= con_data(2 downto 0) & '0';
```

```
    shiftright <= '0' & con_data(3 downto 1);
```

```
    sub <= shiftright + (not shiftright) + 1;
```

```
    output <= sub;
```

```
end Behavioral;
```

فيه input 4 bits هتعمله مرة shift left ومرة shift right وتجمعهم علي بعض ودا الـ output.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
entity shift is
Port ( Data1 : in STD_LOGIC_VECTOR (3 downto 0);
output : out STD_LOGIC_VECTOR (3 downto 0));
end shift;
architecture Behavioral of shift is
Signal right: STD_LOGIC_VECTOR (3 downto 0);
Signal left: STD_LOGIC_VECTOR (3 downto 0);
begin
right <= '0' & Data1 (3 downto 1);
left <= Data1 (2 downto 0) & '0';
output <= right + left;
end Behavioral;
```

معك Data1, Data2 4 bits و op 2 bits .. دول input وال output 4 bits
لو ال op ب 00 يعمل shift left ل Data1 بس ويعرضها
لو ال op ب 01 بجمع Data1 و Data2 وبطلعه
وبرضو نفس الشيء لو 10 و 11.

Entity practical is

Port(Data1, Data2: in std_logic_vector (3 downto 0);

op: in std_logic_vector (1 downto 0);

res: out std_logic_vector (3 downto 0));

End practical;

Architecture Behavioral of practical is

Begin

res <= Data1(2 downto 0) & '0' when op = "00" else --Shift left

Data1 + Data2 when op = "01" else --Addition

Data1 and Data2 when op = "10" else --And gate

Data1(3 downto 2) & Data2(1 downto 0) when op = "11";

--Concatenate upper part of Data1 with lower part of Data2

End Behavioral;

اللي بعد كده دي أسئلة اتبعتلي جتلهم السنة اللي فاتت [2021] (بس
مفيش إجابات) و بيقولي ان جالهم مستوي الأسئلة زينا بنفس الترتيب

اعمل mux و حطه ك component في package انت اللي عاملها بعدها module تاني و port map للمux بالطريقة اللي هو طالبها.

جالي register ليه 32bits input و داخل عليه clear و reset و shift و load و multiply و clk و كل دول bit واحدة و output 32 bits و طالب بقي مرة تعمل Clear و reset و مرة multiply و مرة shift و مرة load تخليهم يطلعوا عال output يعني.

Synchronous d flip flop:

بتعمله implementation زي بتاع اللابات (Lab 3 slide 16 + 17)

Flop register:

زي اللي فال slide بالظبط بس خدوا بالكوا من synchronous و asynchronous عشان نقصت درجتين عشان مخدمتش بالي من a ديه و عملتها بطريقة غير اللي طالبها بس المعيد محترم يعني ونقصني درجتين بس (Lab 3 slide 20 + 21 + 22)

جالي MUX 2x1 عادية جدا تعملها وتحطها في package و تستخدمها ك component في ال main module اللي entity بتاعتها فيها input data واحد و select و output وال input دا وال complement بتاعه هدخلهم في ال mux دا غير ال select وال output طبعاً.

Mux 2x1:

Data1, Data2, output 4 bits, selection 1 bit

بتعمل 2 signals اول واحدة هتجمع فيها ال 2 data وتاني واحده هتعمل and بينهم وتدخلهم على ال Mux (هتدخل ال signals).

جه sign extender 4-bits to 8-bits تحطه في package و تستخدمه
ك component في انك تعمل Extend لرقمين و تطلع مجموعهم و
ال carry لو فيه.

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
package mypack is
component signext is
port(a: in STD_LOGIC_VECTOR(3 downto 0);
y: out STD_LOGIC_VECTOR(7 downto 0));
end component;
end mypack;
```

--Package code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;
use work.mypack.all;
entity Data is
port(a: in STD_LOGIC_VECTOR(3 downto 0);
b: in STD_LOGIC_VECTOR(3 downto 0);
output : out STD_LOGIC_VECTOR(7 downto 0);
carry : out std_logic);
end Data;
architecture Behavioral of Data is
signal sum :std_logic_vector (8 downto 0);
signal out1 , out2 :std_logic_vector (7 downto 0);
begin
m1 : signext port map (a,out1);
m2 : signext port map (b,out2);
sum <= out1 + out2;
```

```
output <= sum(7 downto 0);  
carry <= sum(8);  
end Behavioral;          --Main code
```

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
entity signext is  
port(a: in STD_LOGIC_VECTOR(3 downto 0);  
y: out STD_LOGIC_VECTOR(7 downto 0));  
end signext;  
architecture Behavioral of signext is  
begin  
y <= "1111" & a WHEN a(3) = '1' else  
"0000" & a;  
end Behavioral;          --Sign extender code
```

2 inputs (4 bits)

3 outputs (first output 4 bits, second & third outputs 1 bit)

First output: subtract 2 inputs.

Second output = 1 if first output is negative else clear.

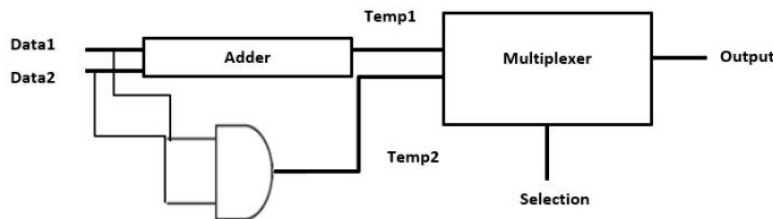
Third output = 1 if first output equals 0 else clear.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
entity p3 is
Port ( data1 : in STD_LOGIC_VECTOR (3 downto 0);
      data2 : in STD_LOGIC_VECTOR (3 downto 0);
      output : out STD_LOGIC_VECTOR (3 downto 0);
      Zeroflag : out STD_LOGIC;
      Isnegative : out STD_LOGIC);
end p3;
architecture Behavioral of p3 is
signal r : STD_LOGIC_VECTOR (3 downto 0);
begin
r <= data1 - data2;
output <= r;
Zeroflag <= '1' when r = "0000" else '0';
Isnegative <= '1' when r(3) = '1' else '0';
end Behavioral;
```

Question

Implement the following module that can:

- Add Data1 to Data2 and store the Summation result in temp1.
- Apply (And) operation between Data1 and Data2 and store the result in temp2.
- Choose to output temp1 or temp2 as shown in the diagram.



The module has the following input/outputs:

Data1: 4-bit vector containing input1.

Data2: 4-bit vector containing input2.

Selection: 1 bit that determines which signal (temp1, temp2) to be output.

Output: 4-bit vector containing the final output.

Control Word

Data1 (4-bit)	Data2 (4 bit)	Selector (1-bit)
------------------	------------------	---------------------

Simulation:

- Input= 1101, 1101, 0 → output = 1010.
- Input=1001, 1100, 1 → output= 0000.
- Input=1101, 1011, 1 → output=1001.

Test Process:

-- Stimulus process

Stim_proc: process

Begin

Data1 <= "1101";

Data2 <= "1101";

Selection <= "0";

Wait for 100ns;

Data1 <= "1001";

Data2 <= "1100";

Selection <= "1";

Wait for 100ns;

Data1 <= "1101";

Data2 <= "1011";

Selection <= "1";

Wait;

End process;

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.STD_LOGIC_unsigned.ALL;

entity addandmux is

Port (data1 : in STD_LOGIC_VECTOR (3 downto 0);

data2 : in STD_LOGIC_VECTOR (3 downto 0);

selection : in STD_LOGIC;

output : out STD_LOGIC_VECTOR (3 downto 0));

end addandmux;

architecture Behavioral of addandmux is

signal temp1,temp2: STD_LOGIC_VECTOR (3 downto 0);

begin

temp1 <= data1 + data2;

temp2 <= data1 and data2;

output <= temp1 when selection <= '0' else temp2;

end Behavioral;

Question:

Implement a sequential circuit consists of a 4 bit counting down counter (with synchronous Set)

With the following characteristic table:

set	Count	CLK	Q
1	X	Trig	"1111"
0	0	Trig	No change
0	1	Trig	Count down

It has the following inputs/outputs:

- Inputs: set, count, clk.
- Output: Q

- Requirement:

- Main module
-

entity qwerty is

```
Port ( set : in STD_LOGIC;
      count : in STD_LOGIC;
      clk : in STD_LOGIC;
      Q : out STD_LOGIC_VECTOR (3 downto 0));
```

end qwerty;

architecture Behavioral of qwerty is

```
signal temp1: std_logic_vector (3 downto 0);
```

```
begin
```

```
process(clk)
```

```
begin
```

```
if(clk' event and clk = '1') then
```

```
    if(set = '1') then
```

```
        temp1 <= "1111";
```

```
    elsif (count = '1') then
```

```
        temp1 <= temp1 - "0001";
```

```
    end if;
```

```
end if;
```

```
end process;
```

```
Q <= temp1;
```

```
end Behavioral;
```

Generic register:

بيدخله input و بتعمل عليه واحدة من ال operations دي:
(Decrement 1, multiple 4, reset, load, shift right)

اللي يعرف إجابة أي مسألة (كود كامل) يبعثلي و احطه

تابعوا البوستات دي بينزل فيها أسئلة تانية

<https://www.facebook.com/groups/671588110417820/posts/1047699869473307/>

<https://www.facebook.com/groups/671588110417820/posts/1048356919407602/>

شوفوا الpdf اونلاين متحمل هوش عشان ممكن اعدل تاني و ازود أسئلة